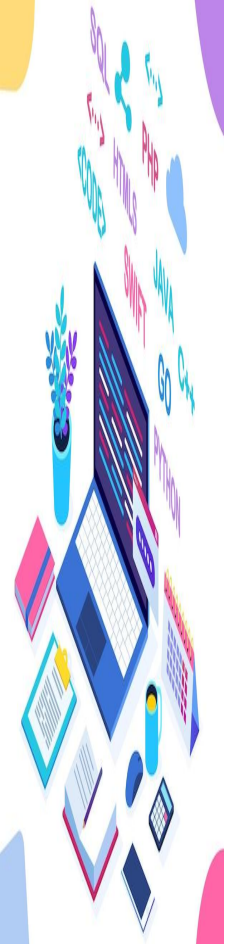


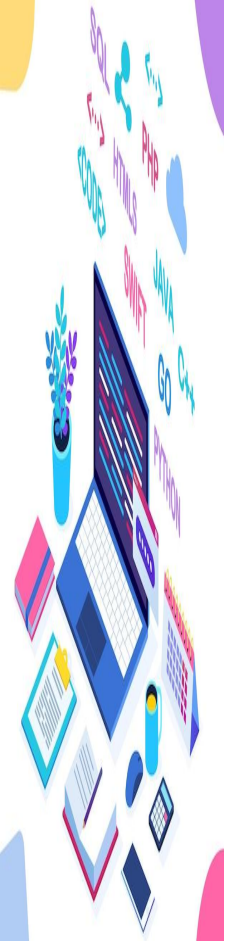
RAJALAKSHMI ENGINEERING COLLEGE

CS23333
Object Oriented Programming Using JAVA



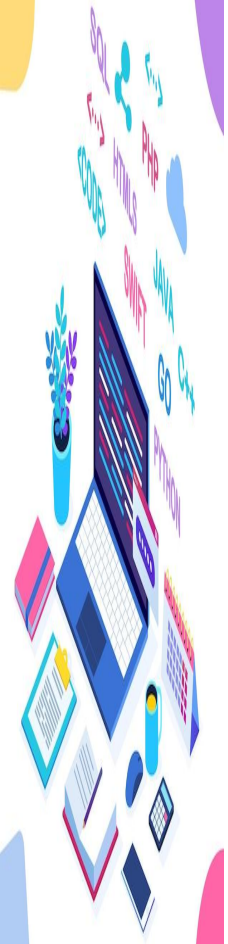
I/O Streams

- Java programs perform I/O through streams. A stream is:
  - – an abstraction that either produces or consumes information
  - – linked to a physical device by the Java I/O system
- All streams behave similarly, even if the actual physical devices to which they are linked differ.
- Thus the same I/O classes can be applied to any kind of device as they abstract the difference between different I/O devices.



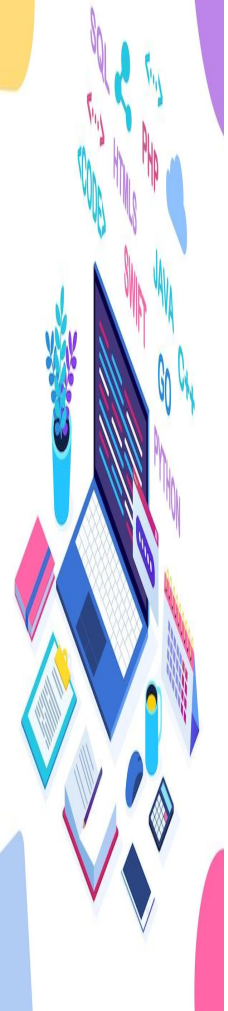
I/O Streams

- Java's stream classes are defined in the java.io package.
- Java 2defines two types of streams:
  - – byte streams
  - – character streams
- Bytestreams:
  - – provide a convenient means for handling input and output of bytes
  - – are used for reading or writing binary data
- Character streams:
  - – provide a convenient means for handling input and output of characters
  - – use Unicode, and, therefore, can be internationalized



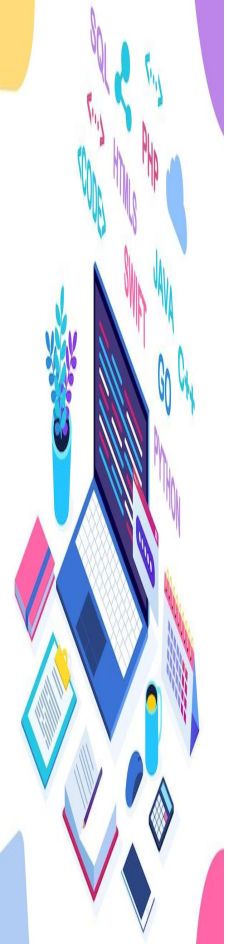
The Predefined Streams

- System class of the java.lang package contains three predefined stream variables, in, out and err.
- These variables are declared as public and static within System:
  - – System.out refers to the standard output stream which is the console.
  - – System.in refers to standard input, which is the keyboard by default.
  - – System.err refers to the standard error stream, which also is the console by default.



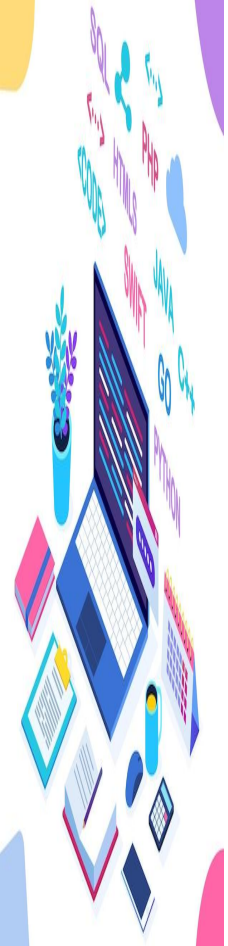
Difference between System.out and System.err

- System.out sends the output to the standard output stream, which is console by default.
- System.err sends the output to the standard error stream, which also happens to be console by default
- The reason behind having two separate streams for output and error is that the standard output should be used for regular program outputs while standard error should be used for error messages



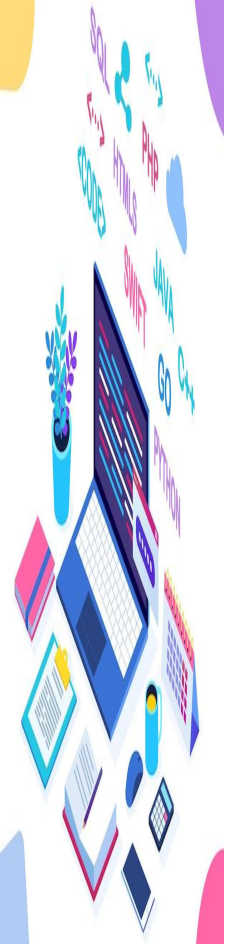
Difference between System.out and System.err

- Both these streams can be redirected to different destinations.
- We can redirect the program output to a particular log file and the error messages to another
- log file by using the following syntax :
- java StreamDemo > output.log 2>error.log
- (where StreamDemo is the name of the class)
- During execution, the program output will be stored in output.log while the error message(if any) will be stored in error.log



Demonstration of System.out and System.err

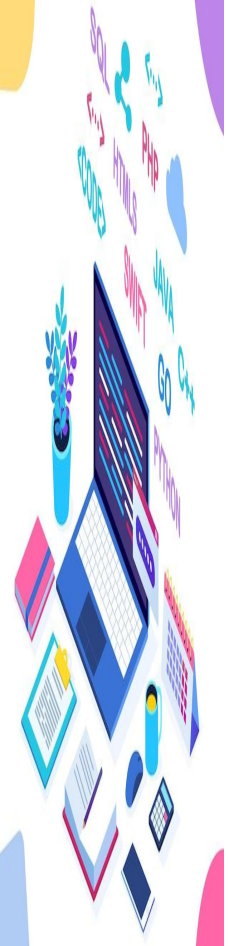
```
class StreamDemo {
 public static void main(String[] args) {
 try {
 System.out.print("Writing program output to the output file ");
 int i=0;
 int z=100/i;
 }
 catch(Exception e) {
 System.err.print("ArithmeticException has occured");
 }
 }
 }
```



System Properties

- System Properties provide information about local system configuration.
- When the Java Virtual Machine starts, it inserts local System Properties into a System properties list.
- We can use methods defined in System class to access or change the values of these properties.

```
public static Properties getProperties()
public static String getProperty(String key)
public static void setProperties(Properties prp)
```

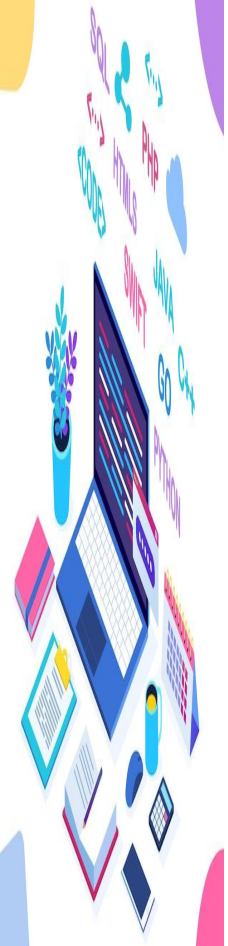


System.getProperties ()

public static Properties getProperties()

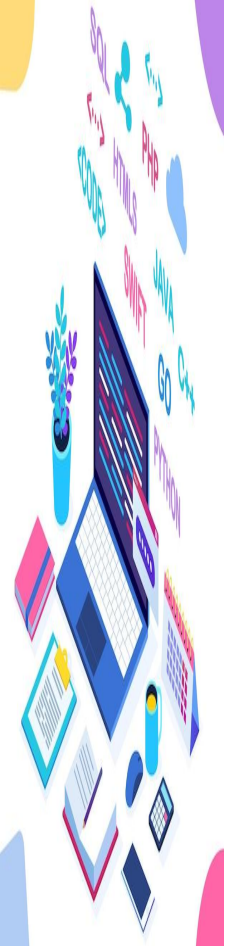
The System.getProperties() method returns an object of the type Properties.

You can use this method to list all the System Properties.



System.getProperty (String Key)- Demo

```
class GetPropertyDemo {
 public static void main(String[] args) {
 String user_home= System.getProperty("user.home");
 String java_version = System.getProperty("java.version");
 String java_home = System.getProperty("java.home");
 String class_path = System.getProperty("java.class.path");
 String os_name = System.getProperty("os.name");
 String user_name = System.getProperty("user.name");
 String user_dir = System.getProperty("user.dir");
 System.out.println("The user home directory is "+user_home);
 System.out.println("The java version is "+java_version);
 System.out.println("The Java Home directory is "+java_home);
 System.out.println("The class path is set to "+class_path);
 System.out.println("The Operating System is "+os_name);
 System.out.println("The user name is "+user_name);
 System.out.println("The working directory is "+user_dir);
 }
 }
```



System.getProperty (String Key)- Demo

Output:

The user home directory is C:\Users\harb

The java version is 1.6.0_05

The Java Home directory is D:\Program Files\java1.6\jdk1.6.0_05\jre

The class path is set to

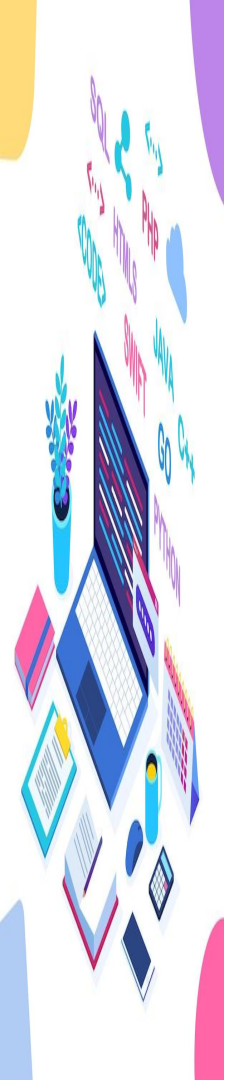
.;E:\app\harb\product\11.1.0\db_1\oui\jlib\classes12.jar;E:\Training\PR

P2012JAVA\Excel to Java\jxl-2.6.9.jar;

The Operating System is Windows Vista

The user name is harb

The working directory is D:\Java\day10\io

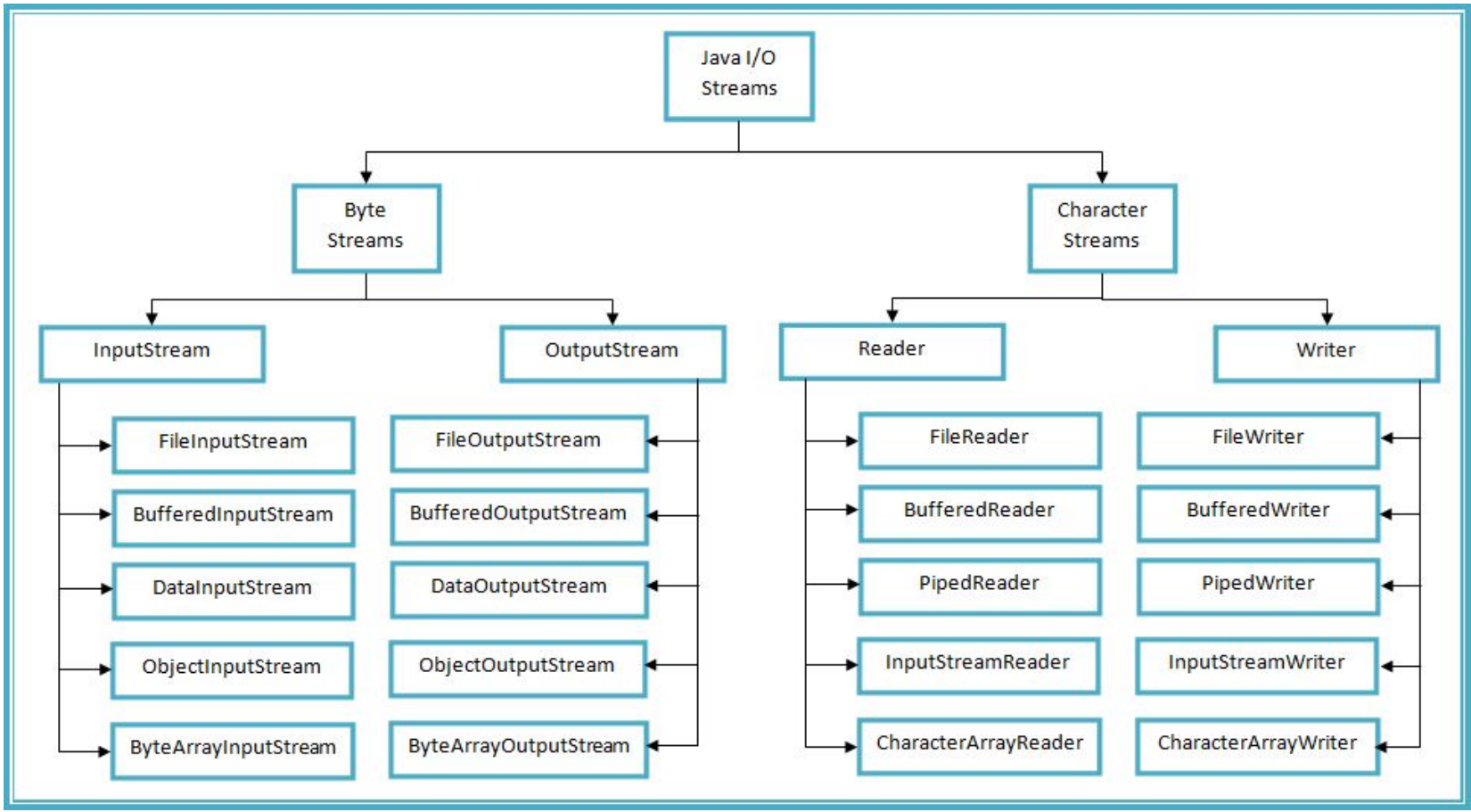


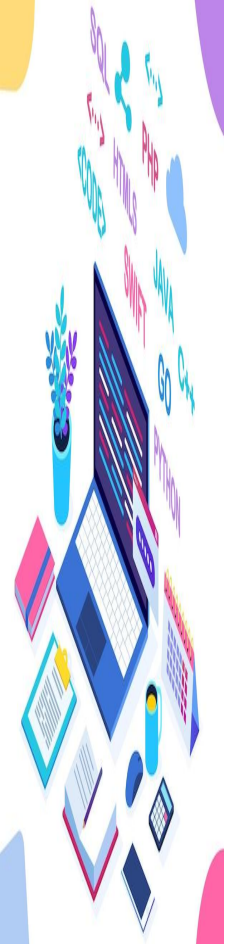
I/O Streams hierarchy

- Java programs perform I/O through streams.
- A stream is an abstraction that either produces or consumes information.
- A stream is linked to a physical device by the Java I/O system.
- All streams behave in the same manner, even if the actual physical devices to which they are linked differ.
- Thus, the same I/O classes and methods can be applied to different types of devices. This means that an input stream can abstract many different kinds of input: from a disk file, a keyboard, or a network socket.
- Likewise, an output stream may refer to the console, a disk file, or a network connection.
- Streams are a clean way to deal with input/ output without having every part of your code understand the difference between a keyboard and a network, for example.
- Java implements streams within class hierarchies defined in the java.io package



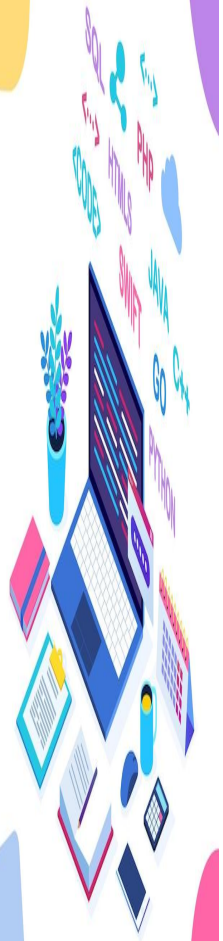
I/O Streams hierarchy





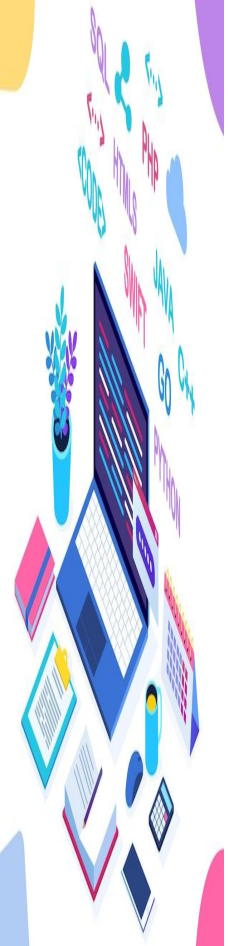
Byte Stream classes

- Byte streams provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- Byte streams are defined by using two class hierarchies. At the top are two abstract classes: InputStream and OutputStream.
- Each of these abstract classes has several concrete subclasses that handle the differences among various devices, such as disk files, network connections, and even memory buffers.



Byte Stream classes

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements **InputStream**
FilterOutputStream	Implements **OutputStream**
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains **print( )** and **println( )**
PushbackInputStream	Input stream that supports one-byte "unget," which returns a byte to the input stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other



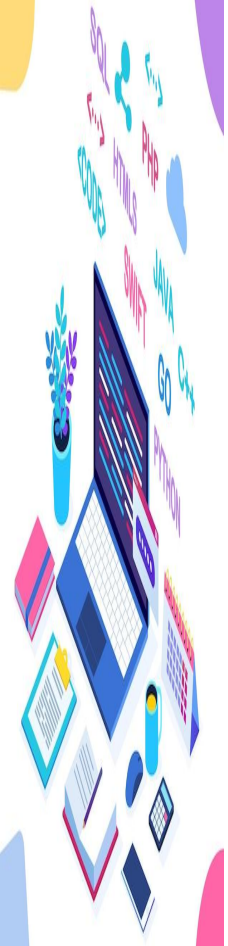
Reading from a File using FileInputStream

```
import java.io.FileInputStream;
import java.io.IOException;

public class SimpleFileRead {
    public static void main(String[] args) throws IOException {
        // Create a FileInputStream to read the file
        FileInputStream fis = new FileInputStream("example.bin");

        int byteRead;
        while ((byteRead = fis.read()) != -1) {
            System.out.print((char) byteRead); // Print each byte as a character
        }

        fis.close(); // Don't forget to close the stream
    }
}
```



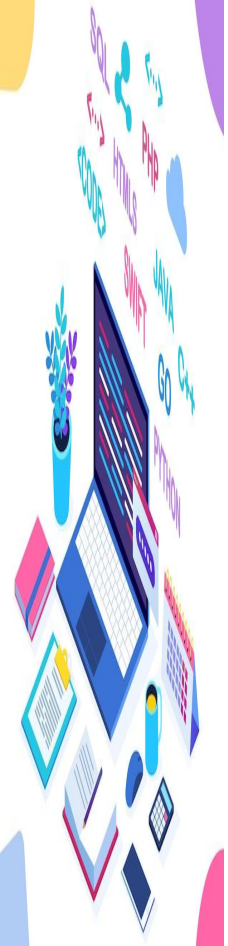
Buffered Input using BufferedInputStream

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class SimpleBufferedRead {
    public static void main(String[] args) throws IOException {
        // Create a BufferedInputStream to read the file with buffering
        BufferedInputStream bis = new BufferedInputStream(new FileInputStream("example.bin"));

        int byteRead;
        while ((byteRead = bis.read()) != -1) {
            System.out.print((char) byteRead); // Print each byte as a character
        }

        bis.close(); // Close the stream
    }
}
```



Reading Primitive Data Types using DataInputStream

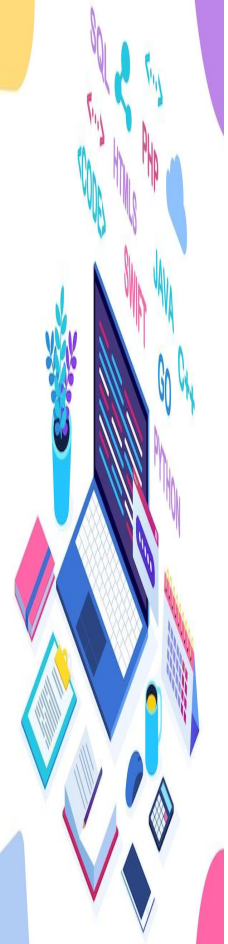
```
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class SimpleDataRead {
    public static void main(String[] args) throws IOException {
        // Create a DataInputStream to read primitive data types
        DataInputStream dis = new DataInputStream(new FileInputStream("example.bin"));

        int i = dis.readInt(); // Read an integer
        double d = dis.readDouble(); // Read a double

        System.out.println("Integer: " + i);
        System.out.println("Double: " + d);

        dis.close(); // Close the stream
    }
}
```



Character Stream classes

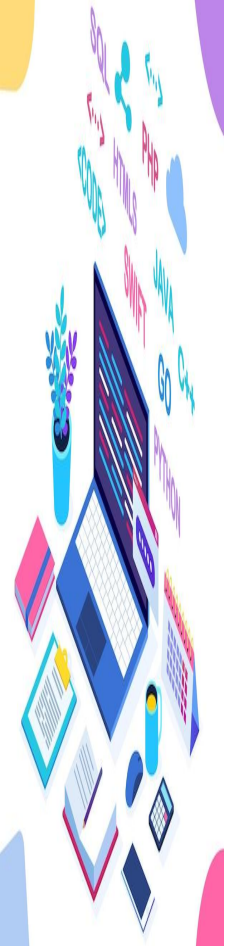
Character streams provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.

The Character Stream Classes Character streams are defined by using two class hierarchies. At the top are two abstract classes: Reader and Writer. These abstract classes handle Unicode character streams. Java has several concrete subclasses of each of these.



Character Stream classes

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains **print( )** and **println( )**
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output



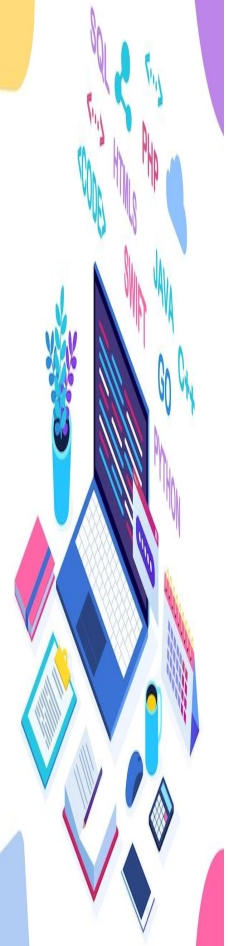
FileReader — Reading Characters from a File

```
import java.io.FileReader;
import java.io.IOException;

public class SimpleFileRead {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("example.txt");

        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }

        fr.close();
    }
}
```



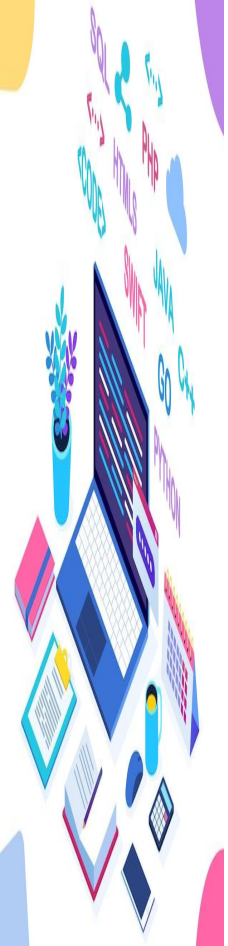
BufferedReader — Efficient Reading of Text

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class SimpleBufferedRead {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader("example.txt"));

        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }

        br.close();
    }
}
```



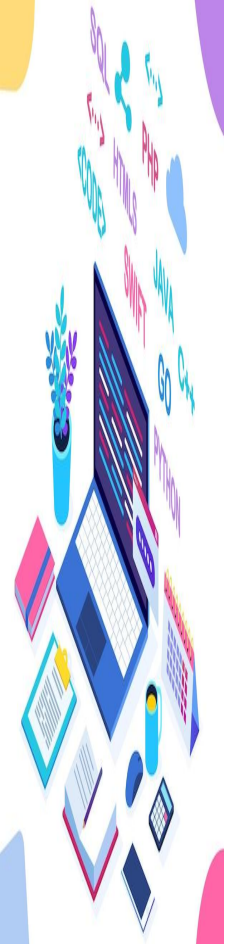
InputStreamReader — Converts Byte Stream to Character Stream

```
import java.io.FileInputStream;
import java.io.InputStreamReader;
import java.io.IOException;

public class SimpleInputStreamReader {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("example.txt");
        InputStreamReader isr = new InputStreamReader(fis);

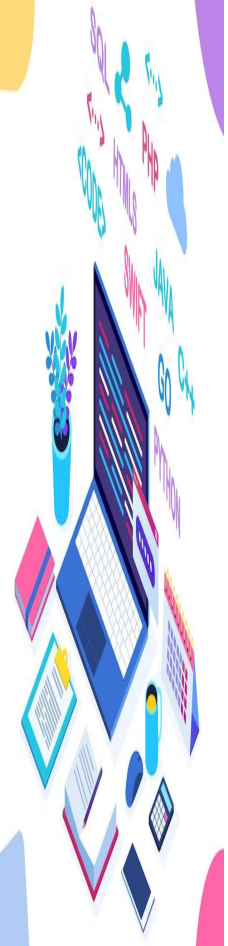
        int ch;
        while ((ch = isr.read()) != -1) {
            System.out.print((char) ch);
        }

        isr.close();
        fis.close();
    }
}
```



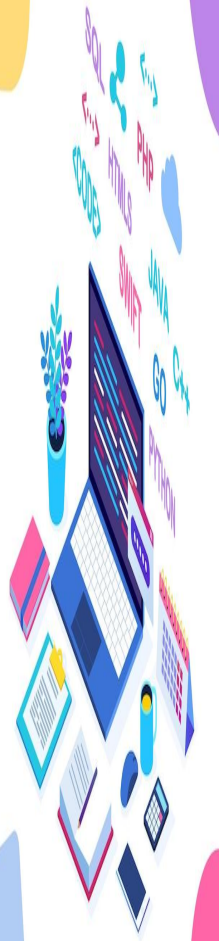
Introduction to Collection

- A Collection is a group of objects
- Collections framework provide a set of standard utility classes to manage collections
- Collections Framework consists of three parts:
  - – Core Interfaces
  - – Concrete Implementation
  - – Algorithms such as searching and sorting

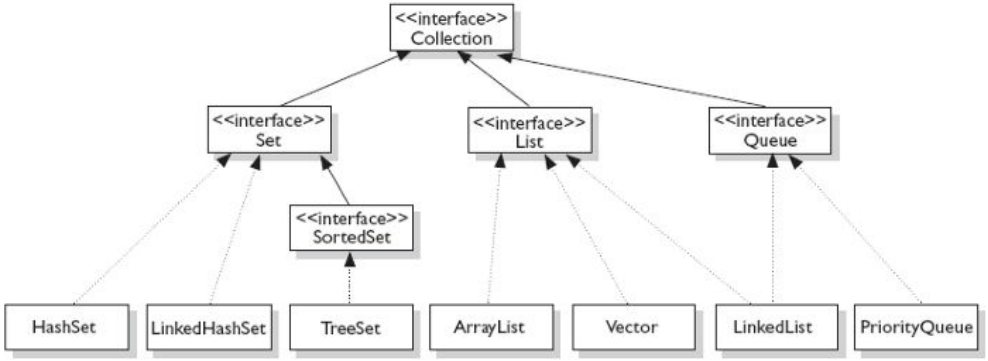


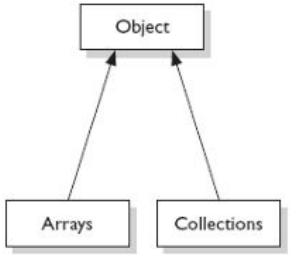
Advantages of Collections

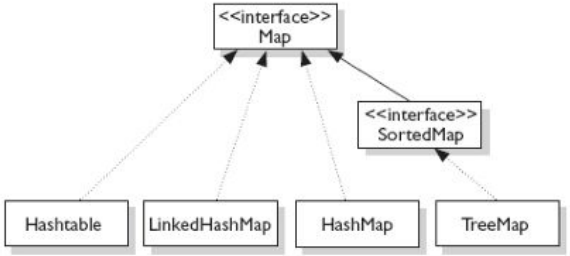
- Reduces programming effort : by providing useful data structures and algorithms so you don't have to write them yourself.
- Increases performance :by providing high-performance implementations of useful data structures and algorithms. Because the various implementations of each interface are interchangeable, programs can be easily tuned by switching implementations.
- Provides interoperability between unrelated APIs: by establishing a common language to pass collections back and forth.
- Reduces the effort required to learn APIs: by eliminating the need to learn multiple ad hoc collection APIs.
- Reduces the effort required to design and implement APIs: by eliminating the need to produce ad hoc collections APIs.
- Fosters Software reuse: by providing a standard interface for collections and algorithms to manipulate them.



Collection Hierarchy
Interfaces and their implementation





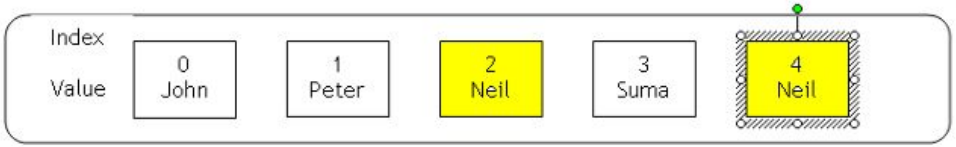


Collection Interfaces

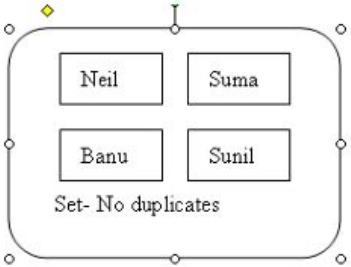


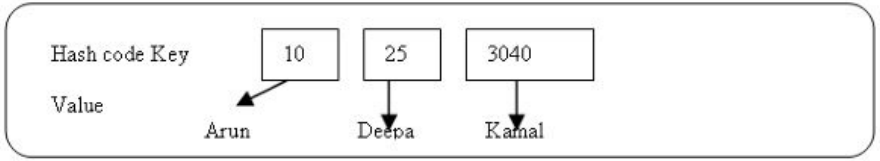
Interfaces	Description
Collection	A basic interface that defines the operations that all the classes that maintain collections of objects typically implement.
Set	Extends the Collection interface for sets that maintain unique element.
SortedSet	Augments the Set interface or Sets that maintain their elements in sorted order.
List	Collections that require position-oriented operations should be created as lists. Duplicates are allowed.
Queue	Things arranged by the order in which they are to be processed.
Map	A basic interface that defines operations that classes that represent mapping of keys to values typically implement.
SortedMap	Extends the Map interface for maps that maintain their mappings in the key order.

Illustration of List, Set and Map



List - Duplicates allowed



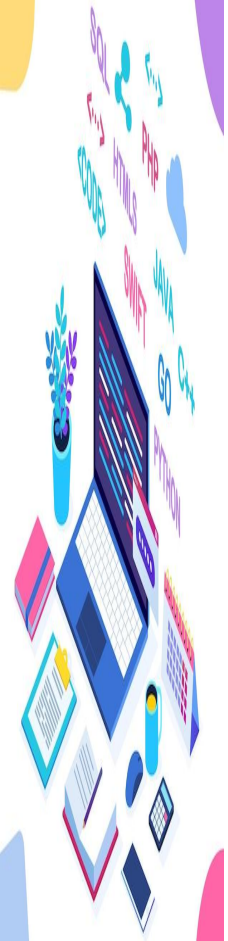


Hash Map - Key generated from RollNo



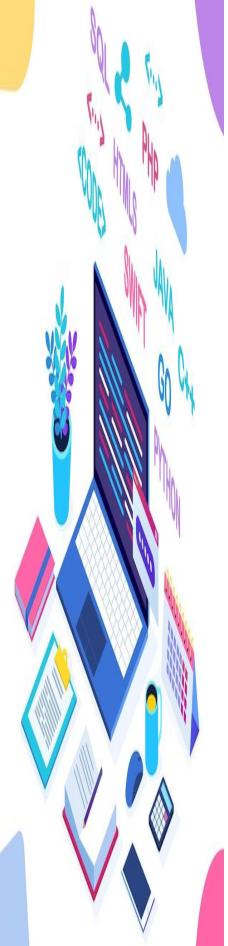
Collection Implementations

		Implementations				
		Hash Table	Resizable Array	Balanced Tree	Linked List	Hash Table + Linked List
Interfaces	Set	HashSet		TreeSet		LinkedHashSet
	List		ArrayList		LinkedList	
	Map	HashMap		TreeMap		LinkedHashMap



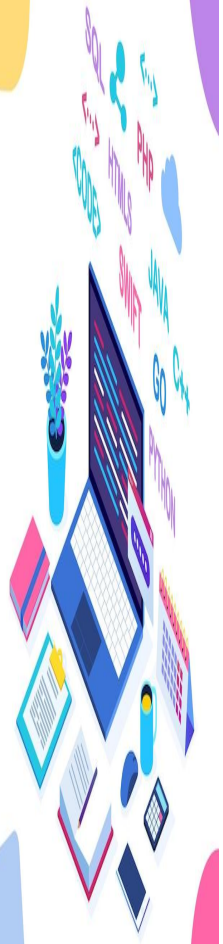
Collection Implementations

- Classes that implement the collection interfaces typically have names of the form <Implementation-style><Interface>.
- The general purpose implementations are summarized in the table above.
- **HashSet** AHashSet is an unsorted, unordered Set. It uses the hashcode of the object being inserted, so the more efficient your hashCode() implementation the better access performance you'll get. Use this class when you want a collection with no duplicates and you don't care about order when you iterate through it.
- **TreeSet** A TreeSet stores objects in a sorted sequence. It stores its elements in a tree and they are automatically arranged in a sorted order



Collection Implementations

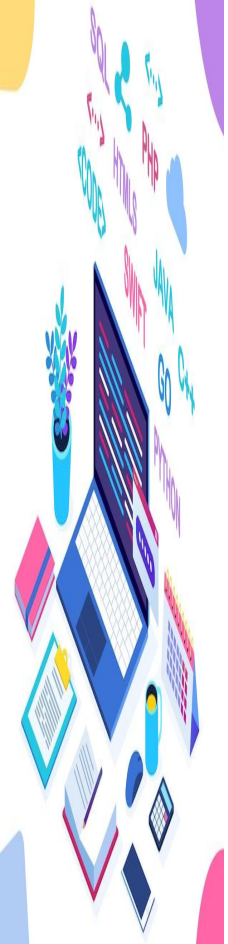
- **LinkedHashSet** A LinkedHashSet is an ordered version of HashSet that maintains a doubly-linked List across all elements. Use this class instead of HashSet when you care about the iteration order. When you iterate through a HashSet the order is unpredictable, while a LinkedHashSet lets you iterate through the elements in the order in which they were inserted.
- **ArrayList** Think of this as a growable array. It gives you fast iteration and fast random access. It is an ordered collection (by index), but not sorted. ArrayList now implements the new RandomAccess interface—a marker interface (meaning it has no methods) that says, "this list supports fast (generally constant time) random access." Choose this over a LinkedList when you need fast iteration but aren't as likely to be doing a lot of insertion and deletion.
- **LinkedList** A LinkedList is ordered by index position, like ArrayList, except that the elements are doubly-linked to one another



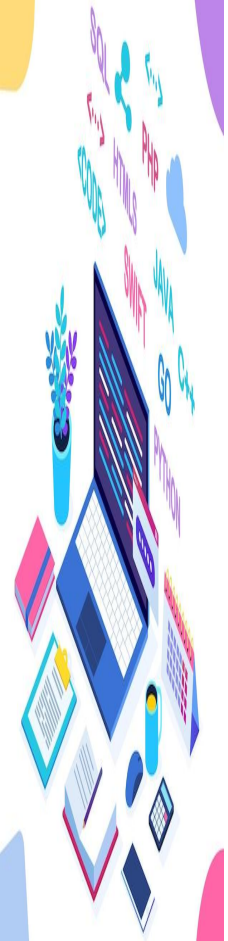
Collection interface methods

Method	Description
`int size();`	Returns number of elements in collection.
`boolean isEmpty();`	Returns true if invoking collection is empty.
`boolean contains(Object element);`	Returns true if element is an element of invoking collection.
`boolean add(Object element);`	Adds element to invoking collection.
`boolean remove(Object element);`	Removes one instance of element from invoking collection
`Iterator iterator();`	Returns an iterator fro the invoking collection
`boolean containsAll(Collection c);`	Returns true if invoking collection contains all elements of *c*; false otherwise.
`boolean addAll(Collection c);`	Adds all elements of *c* to the invoking collection.
`boolean removeAll(Collection c);`	Removes all elements of *c* from the invoking collection
`boolean retainAll(Collection c);`	Removes all elements from the invoking collection except those in *c*.
`void clear();`	Removes all elements from the invoking collection
`Object[] toArray();`	Returns an array that contains all elements stored in the invoking collection
`Object[] toArray(Object a[]);`	Returns an array that contains only those collection elements whose type matches that of *a*.

List



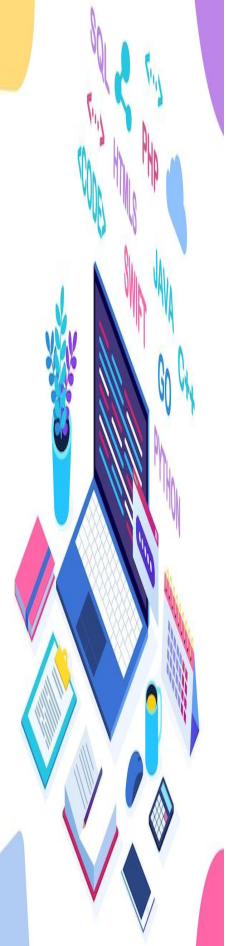
- List interface extends from Collection interface
- It stores elements in a sequential manner
- Elements in the list can be accessed or inserted based on their position
- Starts with zero based index
- Can contain duplicate elements
- An Iterator can be used to access the elements of the List



The ArrayList Class

- ArrayList class implements List interface
- It supports dynamic array that can grow dynamically
- Standard arrays are of fixed size. After arrays are created they cannot grow or shrink
- It provides more powerful insertion and search mechanisms than arrays
- Gives faster Iteration and fast random access
- Ordered Collection (by index), but not Sorted

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(0, new Integer(42));
int total = list.get(0).intValue();
```

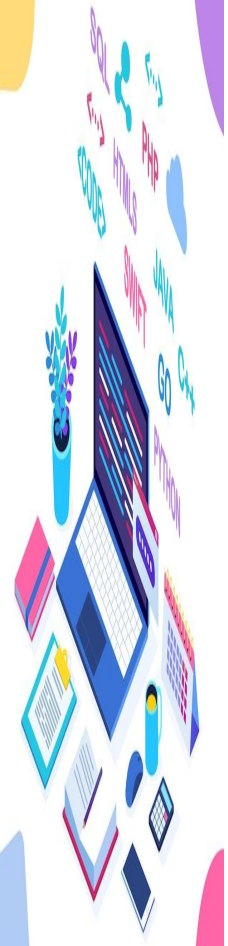


Example

```
import java.util.*;
public class ArrayListTest {
public static void main(String[] args) {
List<String> test = new ArrayList<String>();
String s = "hi";
test.add("string");
test.add(s);
test.add(s+s);
System.out.print(test.size());
System.out.print(test.contains(42));
System.out.print(test.contains("hihi"));
test.remove("hi");
System.out.print(test.size());
} }
```

which produces:

3 false true 2

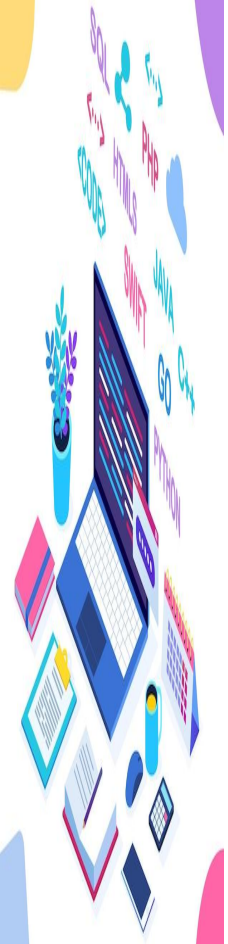


Iterator

- Iterator is an object that enables you to traverse through a collection
- Can be used to remove elements from the collection selectively, if desired

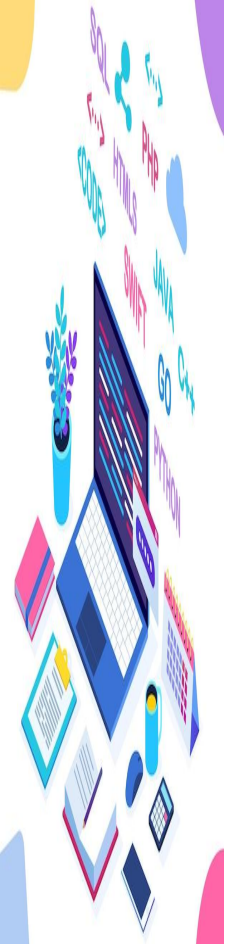
```
public interface Iterator<E>
 {
 boolean hasNext();
 E next();
void remove();
}
```

```
ArrayList<Integer> ai=new ArrayList<Integer>();
 Iterator i=ai.iterator();
 while (i.hasNext())
 System.out.println(i.next());
```



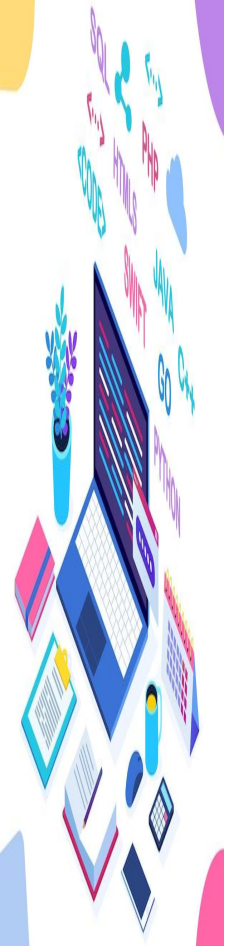
Iterator

- boolean hasNext()- Returns true if there are more elementsObject
- next()- Returns next element. Throws NoSuchElementException if there is no next element.
- void remove()- Removes current element. Throws IllegalStateException if an attempt is made to call remove() that is not preceded by a call to next()



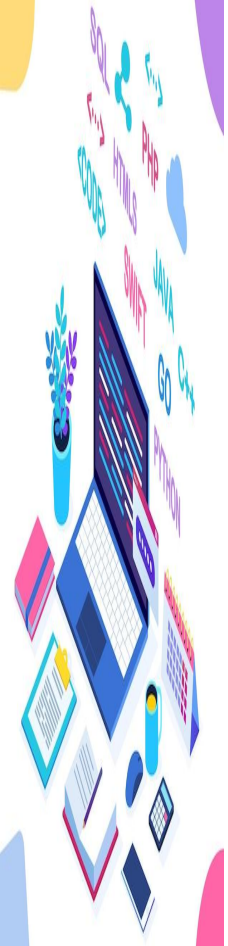
ListIterator

- Used for obtaining a iterator for collections that implement List
- ListIterator gives us the ability to access the collection in either forward or backward direction
- Has both next() and previous() method to access the next and previous element in the List



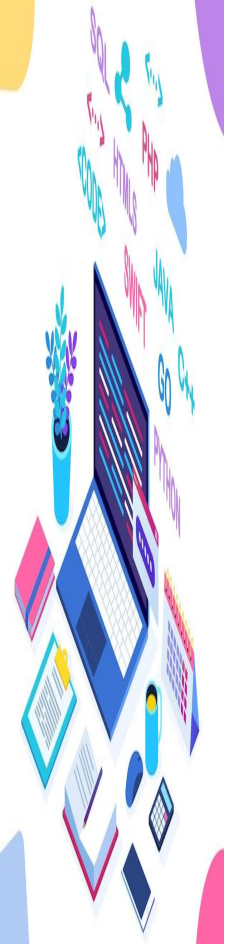
Example

```
class ListIteratorExample {
 public static void main(String[] args) {
 ArrayList aList = new ArrayList();
 //Add elements to ArrayList object
 aList.add("1");
 aList.add("2");
 aList.add("3");
 ListIterator listIterator = aList.listIterator();
 System.out.println("Previous Index is : " + listIterator.previousIndex());
 System.out.println("Next Index is : " + listIterator.nextIndex());
 //advance current position by one using next method
 listIterator.next();
 System.out.println("After increasing current position by one element : ");
 System.out.println("Previous Index is : " + listIterator.previousIndex());
 System.out.println("Next Index is : " + listIterator.nextIndex());
 }}
```



Linked List

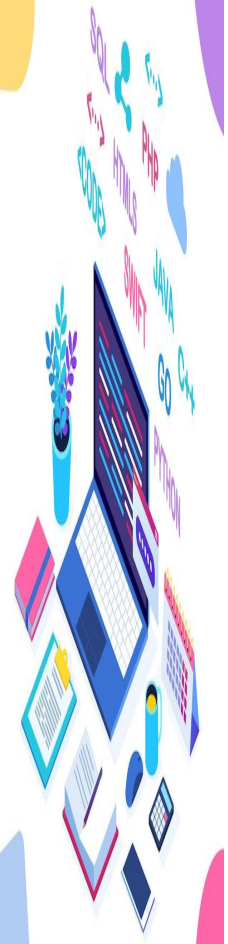
- Implements the List and also the Queue interface
- Some Useful Methods
  - void addFirst(Object x)
  - void addLast(Object x)
  - Object getFirst()
  - Object getLast()
  - Object removeFirst()
  - Object removeLast()



The Vector Class

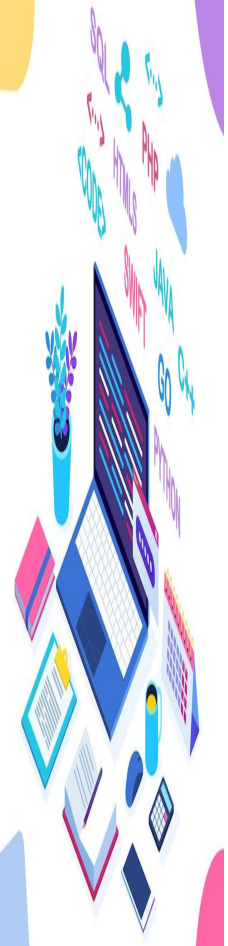
- The java.util.Vector class implements a growable array of Objects
- Same as ArrayList, but Vector methods are synchronized for thread safety
- New java.util.Vector is implemented from List Interface
- Creation of a Vector

```
Vector v1 = new Vector(); // allows old or new methods
List v2 = new Vector(); // allows only the new (List) methods.
```



Map Interface

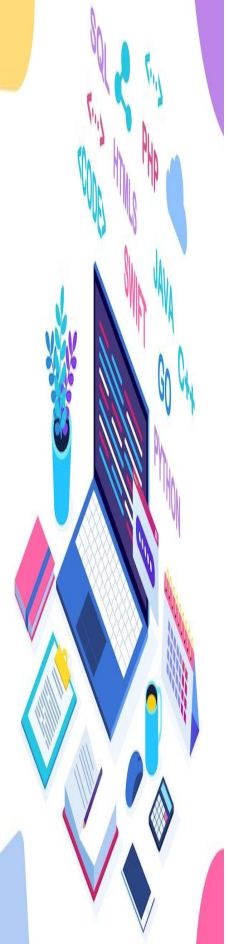
- Map is an interface that stores data in the form of key-value pair
- All the keys in the map will be unique
- We can retrieve the value stored in a map by providing the key value
- A Map cannot contain duplicate values
- Each key can map to at most one value
- For basic operations it uses the following methods
  - put()- for adding elements
  - get()- for retrieving an element
  - remove()- to remove an element
  - size()- to check the size of the Collection



The HashMap class

- HashMap uses the hashcode value of an object to determine how the object should be stored in the collection
- Hashcode is used again to help locate the object in the collection
- Gives you an unsorted and unordered Map
- Allows only one null key and multiple null values in a collection

```
HashMap<String,Double> hm = new HashMap<String,Double>();
hm.put("John Doe", new Double(3434.34));
hm.put("Tom Smith", new Double(123.22));
```

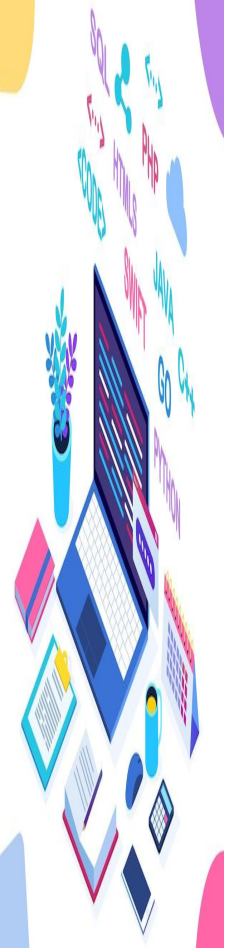


The HashMap class

Map is an object that stores key/value pairs.

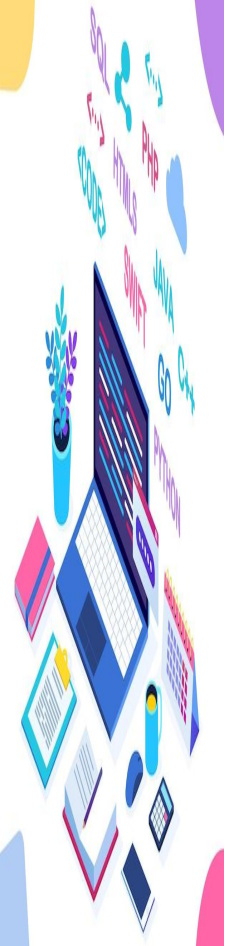
- Given a key, you can find its value. Keys must be unique, but values may be duplicated.
- TheHashMap class provides the primary implementation of the map interface.
- TheHashMap class uses a hash table to implement Map interface.
- This allows the execution time of basic operations, such as get() and put() to be constant.

In the following example, it maps names to account balances.



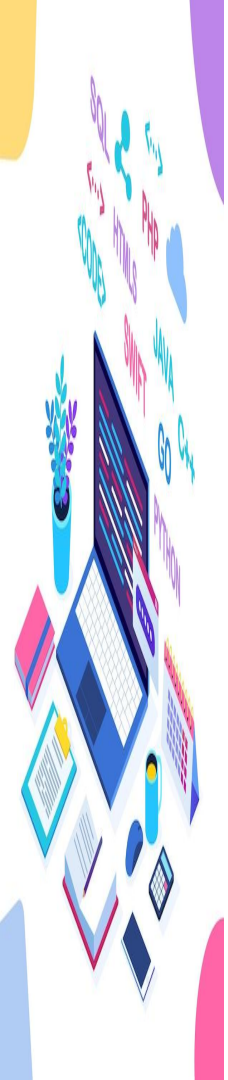
The HashMap class

```
import java.util.*;
 class HashMapDemo {
 public static void main(String args[]) {
 // Create a hash map
 HashMap<String,Double> hm = new HashMap<String,Double>();
 // Put elements to the map
 hm.put("John Doe", new Double(3434.34));
 hm.put("Tom Smith", new Double(123.22));
 hm.put("Jane Baker", new Double(1378.00));
 hm.put("Tod Hall", new Double(99.22));
 hm.put("Ralph Smith", new Double(-19.08));
 // Get a set of the entries
 Set set = hm.entrySet();
```



The HashMap class

```
// Get an iterator
Iterator i = set.iterator();
// Display elements
while(i.hasNext()) {
Map.Entry me = (Map.Entry)i.next();
System.out.println(me.getKey() + ": "+ me.getValue()); }
// Deposit 1000 into John Doe's account
double balance =  ((Double)hm.get("John Doe")).doubleValue();
hm.put("John Doe", new Double(balance + 1000));
System.out.println("John Doe's new balance: " + hm.get("John Doe")); } }
```



The HashMap class

The output of the program is

Ralph Smith: -19.08

Tom Smith: 123.22

John Doe: 3434.34

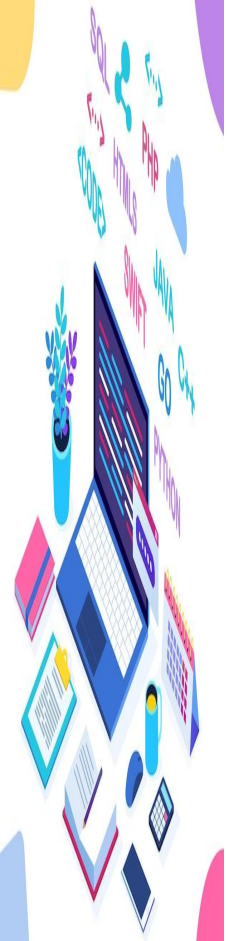
Tod Hall: 99.22

Jane Baker: 1378.0

John Doe's new balance: 4434.34

The above program first populates the HashMap object. Then the contents of the map are displayed using a set-view, obtained by calling entrySet(). The keys & values are displayed by calling getKey() and getValue() methods of the Map.Entry interface.

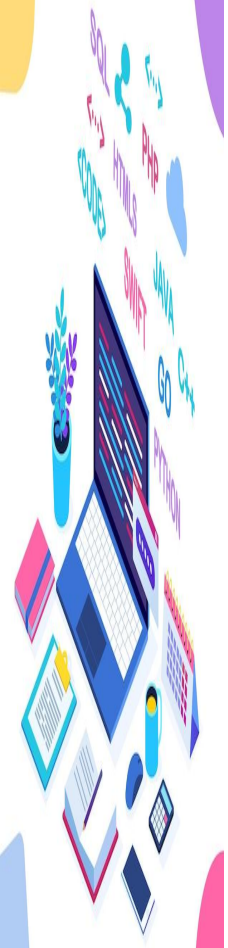
Note: TreeMap instead of HashMap would have given a sorted output



The Hashtable Class

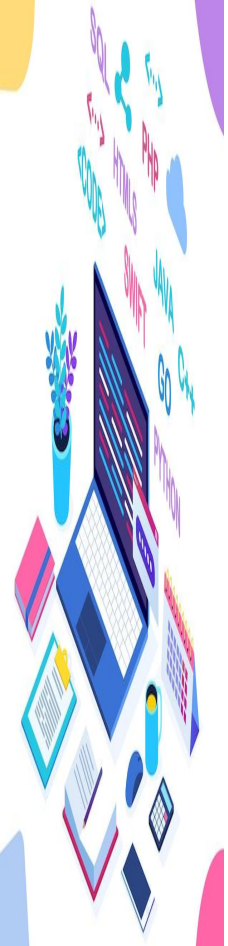
- Part of java.util package
- It implements Map interface and extends Dictionary Class
- It can contain only unique elements
- The key cannot have a null value
- It is a synchronized class

```
Hashtable<String,Double> balance = new Hashtable<String,Double>();
balance.put("Arun", new Double(3434.34));
balance.put("Radha", new Double(123.22));
```



The Hashtable Class

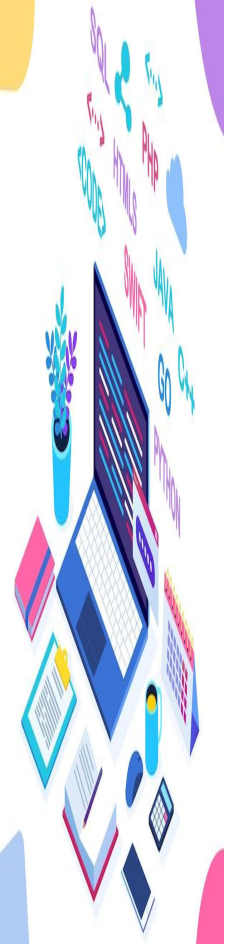
```
import java.util.*;
 class HashTableDemo {
 public static void main(String args[]) {
 // Create a hash map
 Hashtable<String,Double> balance = new Hashtable<String,Double>();
 Enumeration<String> names;
 //String str;
 //double bal;
 balance.put("Arun", new Double(3434.34));
 balance.put("Radha", new Double(123.22));
 balance.put("Ram", new Double(99.22));
```



The Hashtable Class

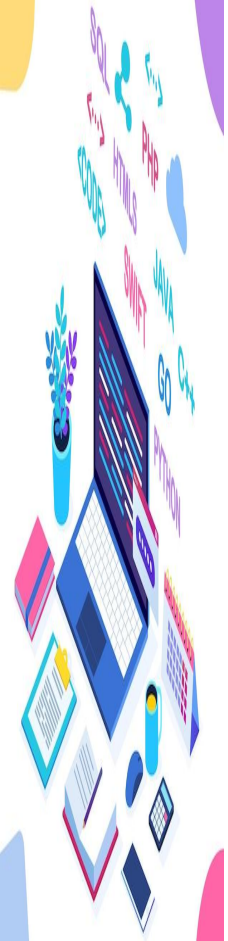
```
// Show all balances in hash table.
 names = balance.keys();
 while(names.hasMoreElements()) {
 String name = names.nextElement();
 System.out.println(name + ": " + balance.get(name));
}
 System.out.println();

 } }
```



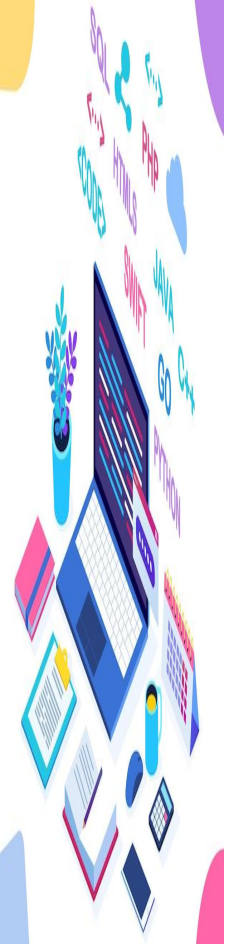
TreeMap

- Implements Map interface
- Provides efficient means of storing key/value pairs in sorted order
- Allows rapid retrieval
- Guarantees that its elements will be sorted in ascending key order
- The Key cannot be null but it can contain multiple null values



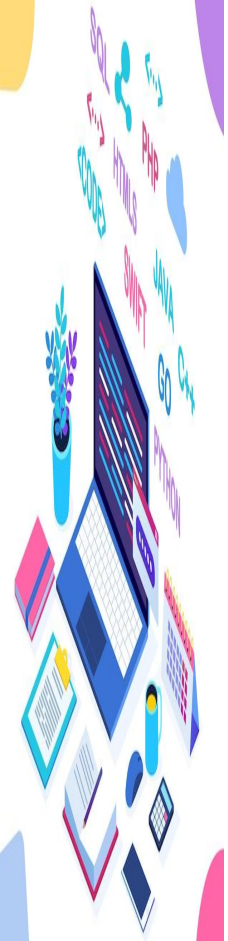
TreeMap

```
import java.util.*;
 class TreeMapDemo{
 public static void main(String arg[]){
 TreeMap tm = new TreeMap();
 tm.put("Suresh",new Double(15357.75));
 tm.put("Meenu",new Float(18345.50));
 tm.put("Viren",new Integer(20000));
 tm.put("Avinash",new Double(19900.25));
 tm.put("Priya",new Integer(12000));
 tm.put("Zakir",new Float(16500.90));
 tm.put("Nirav",new Double(22000));
 tm.put("Jayesh",new Integer(15000));
 tm.put("Poorva","Zero");
```



TreeMap

```
Set salary = tm.entrySet();
 Iterator it = salary.iterator();
 while(it.hasNext()){
 Map.Entry e = (Map.Entry) it.next();
 System.out.println(e.getKey()+" : "+" is "+e.getValue());
 }
} }
```



Properties

Avinash : is 19900.25

Jayesh : is 15000

Meenu : is 18345.5

Nirav : is 22000.0

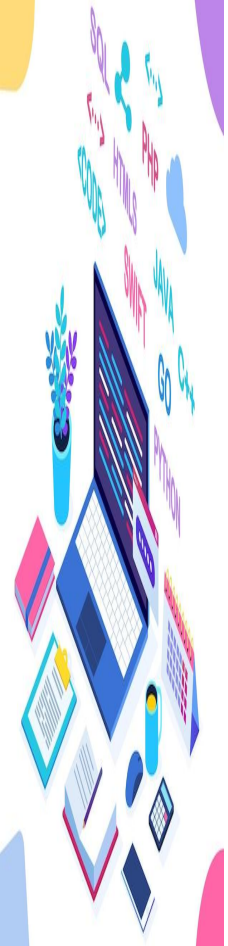
Poorva : is Zero

Priya : is 12000

Suresh : is 15357.75

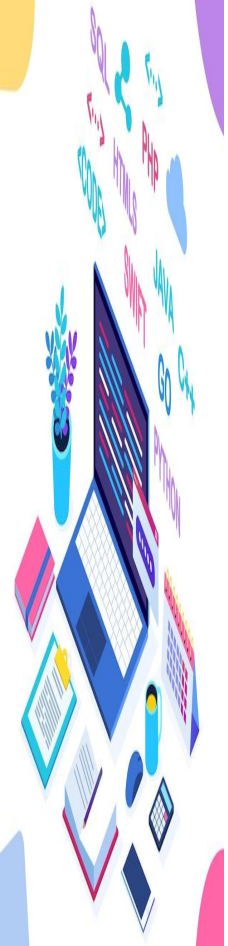
Viren : is 20000

Zakir : is 16500.9



Set

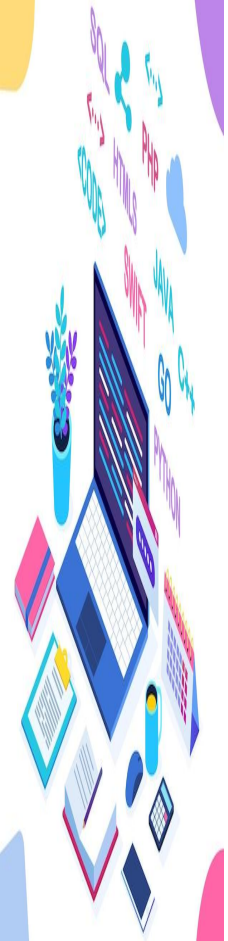
- Set interface extends from Collection interface
- Set is an unordered collection
- It doesn't allow duplicates
- If you add duplicates, it will replace the existing one
- It allows you to add only a single null value
- An iterator can be used to traverse through the list
- Implementation classes for Set are TreeSet, HashSet and LinkedHashSet



TreeSet

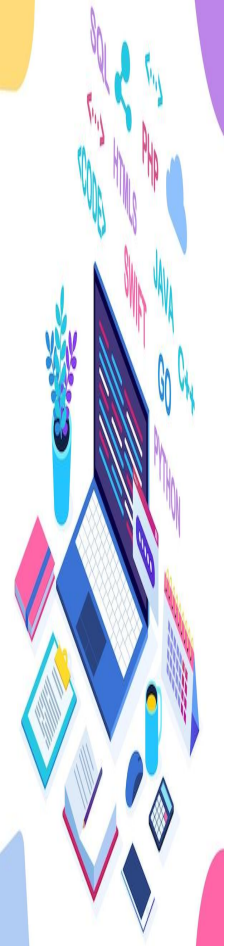
- No duplicates are allowed
- Iterates in sorted order
- Sorted Collection : By default elements will be in ascending order
- Not synchronized : If more than one thread wants to access it at the same time then it must be synchronized externally

```
TreeSet<String> t1 = new TreeSet<String>();
t1.add("One");
t1.add("Two");
t1.add("Three");
```



TreeSet

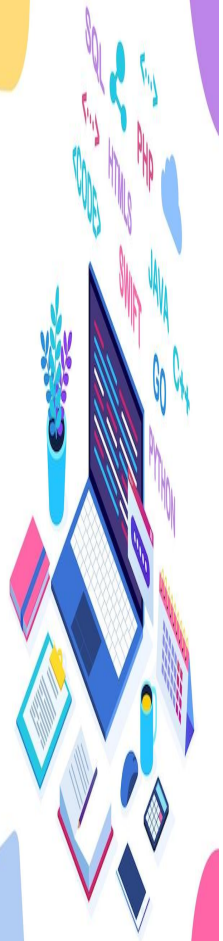
```
class Treeset{
 public static void main(String args[]){
 TreeSet<String> t1 = new TreeSet<String>();
 t1.add("One");
 t1.add("Two");
 t1.add("Three");
 t1.add("Four");
 t1.add("Five");
 System.out.println("Contents of treeset");
 Iterator it1 = t1.iterator();
while(it1.hasNext()){
 Object o1=it1.next();
 System.out.print(o1+"\t");
}
} }
```



TreeSet

Contents of TreeSet:

Five Four One Three Two

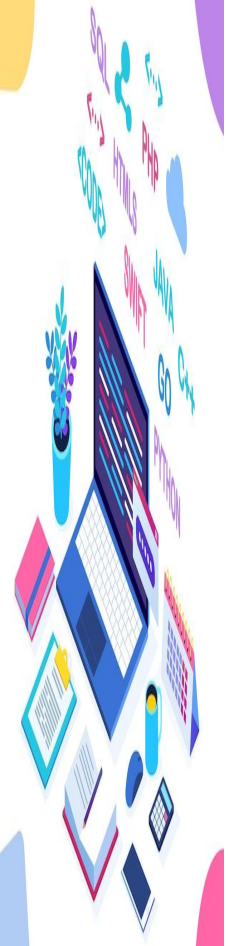


The HashSet Class

- Noduplicates are allowed
- A HashSet is an unsorted, unordered Set
- Can be used when you want a collection with no duplicates and you don't care about the order when you iterate through it
- Uses HashTable for storage

```
Set<Integer> s = new HashSet<Integer>();
ba[0] = s.add(1);
ba[1] = s.add(2);
ba[2] = s.add(3);
```

Remember that Sets are used when you don't want any duplicates in your collection. If you attempt to add an element to a set that already exists in the set, the duplicate element will not be added, and the add() method will return false. Remember, HashSets tend to be very fast because they use hashcodes.



Example

```
import java.util.*;
 class eg {
 public static void main(String[] args) {
 Set<Integer> s = new HashSet<Integer>();
 s.add(1);
 s.add(2);
 s.add(3);
 s.add(4);
s.add(5);
 for(Integer i : s)
 System.out.print(i + " "); }}
```

O/P: 2 4 1 3 5

Note: The order of the objects printed are not predictable